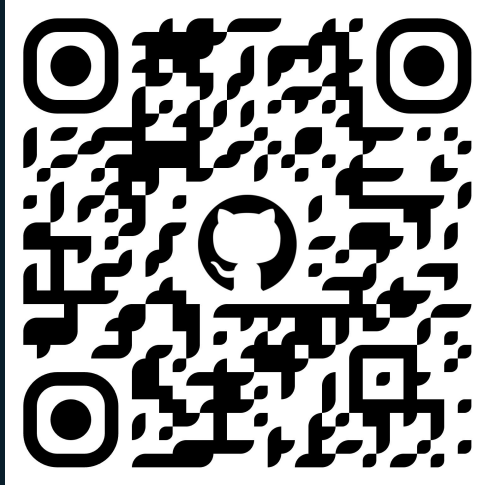


Resource



github.com/ro-witthawin/TBS-BI303-2026

TBS-BI303-2026



Class schedule

Date	Topic	Material
2026-08-03	Intro to Programming Language	Day01
2026-08-10	Basic String & List	Day02
2026-08-17	Function	Day03
2026-08-24	Loop	Day04
2026-08-31	Dictionary	Day05
2026-09-07	Class Object	Day06
2026-09-14	Freq Library	Day07
2026-09-21	Final Exam	

“List”

[. . .]

List

[0, 1, 2, “a”, “B”]

List

[0, 1, 2, “a”, “B”]

Index	[0]	[1]	[2]	[3]	[4]
-------	-----	-----	-----	-----	-----

Negative Index	[-5]	[-4]	[-3]	[-2]	[-1]
-------------------	------	------	------	------	------

Empty List

`[]`



List indexing

```
sample = ["apple", "orange", "pen"]
```

```
sample[0] # "apple"
```

```
sample[-1] # "pen"
```

```
sample[0: 2] # ["apple", "orange"]
```

```
sample[1:] # ["orange", "pen"]
```

```
sample[1: -1] # ["orange"]
```

List Methods

```
sample = ["apple", "orange", "pen"]
```

```
sample.append(value) # Add one item to the end
```

```
sample.insert(index, value) # Add one item at a specific position
```

```
sample.extend(list) # Add multiple items from another list
```

```
sample.remove(value) # Remove a specific value
```

```
sample.pop(index) # Remove an item by index and return it
```

```
sample.clear() # Remove all items
```

List : Example Case #1 Attendance Check

```
checkin = ["A", "B", "C", "D"]
```

```
checkout = ["A", "B", "C"]
```

```
# Find "D" or Find ["A", "B", "C"]
```

```
# Find
```


List : Example Case #2 Attendance Check

```
checkin = [  
    ["A", "09:00"],  
    ["B", "09:01"],  
    ["A", "09:03"]  
]
```

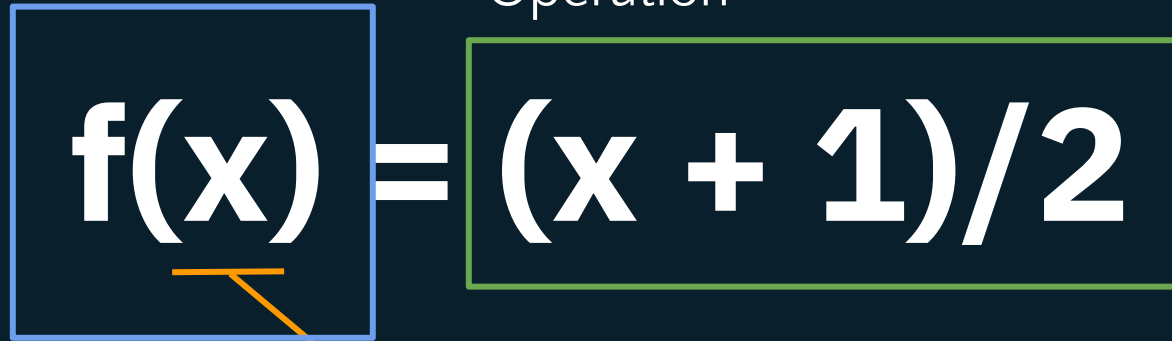
Find average of "A"

“Function”

$f(x)$

Function

Operation



The diagram shows the function definition $f(x) = (x + 1)/2$. The expression $f(x)$ is enclosed in a blue rectangular box, and the expression $(x + 1)/2$ is enclosed in a green rectangular box. An equals sign is placed between the two boxes. An orange line points from the word "Input" to the variable x in the function notation. A short orange horizontal line is positioned directly below the x .

$$f(x) = (x + 1)/2$$

Input

Function in Math

$$v = v_0 + a * t^2$$

$$\text{speed} = \text{init_speed} + \text{accelerate} * \text{time}^{**2}$$

Function in Math

$$v = v_0 + a * t^2$$

$$\text{speed} = \text{init_speed} + \text{accelerate} * \text{time}^{**2}$$

speed(init_speed, accelerate, time) = init_speed + accelerate * time**2

Input

Function

Function in Python

$$v_t = v_0 + a * t^2$$

`speed(init_speed, accelerate, time) = init_speed + accelerate * time**2`

```
def speed(init_speed, accelerate, time):  
    output_value = init_speed + accelerate * time**2  
    return output_value
```

`v_t = speed(0, 9.8, 10)` ✓

`v_t = speed(init_speed = 0, accelerate = 9.8, time =10)` ✓

Function in Python

$$v_t = v_0 + a * t^2$$

`speed(init_speed, accelerate, time) = init_speed + accelerate * time**2`

```
def speed(init_speed, accelerate, time):  
    output_value = init_speed + accelerate * time**2  
    return output_value
```

`v_t = speed(0, 9.8, 10)` ✓

`v_t = speed(init_speed = 0, accelerate = 9.8, time = 10)` ✓

Can we define as default value?

Function in Python

$$v_t = v_0 + a * t^2$$

```
def speed(time, init_speed = 0, accelerate = 9.8):  
    output_value = init_speed + accelerate * time**2  
    return output_value
```

`v_t = speed(10)` ✓

`v_t = speed(10, 9.8)` ✓ # `t=10, v_0=9.8, a=9.8`

`v_t = speed(10, 0, 9.8)` ✓ # `t=10, v_0=0, a=9.8`

`v_t = speed(init_speed = 0, accelerate = 9.8, time = 10)` ✓

`v_t = speed(time = 10, init_speed = 0, accelerate = 9.8)` ✓

Function Condition

$$f(x) = \begin{cases} x + 1, & x > 0 \\ x - 1, & x < 0 \end{cases}$$

Function Condition

$$f(x) = \begin{cases} x + 1, & x > 0 \\ x - 1, & x < 0 \end{cases}$$

```
def eq1(x):  
    if x > 0:  
        output_value = x + 1  
    elif x < 0:  
        output_value = x - 1  
    return output_value
```

```
def eq1(x):  
    if x > 0:  
        return x + 1  
    elif x < 0:  
        return x - 1
```

Hand-on 10 min without AI



$$v(t) = \begin{cases} v_{max}, & v_t \geq v_{max} \\ v_0 + at^2, & \text{other} \end{cases}$$

Settings

- | | | |
|---------------|---|---|
| Site | > | ☒ Show AI-powered inline completions |
| Editor | > | ☒ Consented to use generative AI features |
| AI assistance | > | ☒ Hide generative AI features |

Colab Pro > Gemini can make mistakes, so double-check responses and [use code with caution](#).

GitHub >

Miscellaneous >

```
def speed(...):
```

```
·  
·  
·
```

```
return ...
```

Level : ★☆☆☆☆

Function is more than just Math

```
def imageGenerate(prompt):  
  
    new_prompt = explainPrompt(prompts)  
  
    init_image = text2image(new_prompt)  
  
    reconstruct_image = diffusion(init_image)  
  
    final_image = upscale(reconstruct_image)  
  
    return final_image
```

Sometime we don't need output

```
def imageGenerate(prompt):  
  
    new_prompt = explainPrompt(prompts)  
  
    init_image = text2image(new_prompt)  
  
    reconstruct_image = diffusion(init_image)  
  
    final_image = upscale(reconstruct_image)  
  
    return final_image saveImage(final_image)
```

Sometime function can return more

```
def splitFullName(fullname): # Expect like "Witthawin s"  
    name = fullname.lower().strip() # "witthawin s"  
    first_space_index = name.find(" ") # 9  
    space_count = name.count(" ") # 3  
    firstname = name[:first_space_index] # "witthawin s"[:9]  
    lastname = name[first_space_index+space_count:] # "witthawin s"[12:]  
  
    return firstname, lastname
```

```
firstname, lastname = splitFullName("Witthawin S.")  
# firstname = "witthawin", lastname = "s."
```

Argument setup

```
def function_name(input_1: str,  
                  input_2: int,  
                  input_3: float,  
                  input_4: str,  
                  input_5: list,  
                  input_6 = None,  
                  input_7: int | None = None  
                  ):  
    pass # do-something
```

None Type

None

Any Function without return any value, it will return None

Function without return

```
def imageGenerate(prompt: str):  
  
    new_prompt = explainPrompt(prompts)  
    init_image = text2image(new_prompt)  
    reconstruct_image = diffusion(init_image)  
    final_image = upscale(reconstruct_image)  
    saveImage(final_image)  
  
imageGenerate("Bird with Pen") # Return None
```

Function without return

$$f(x) = \begin{cases} x + 1, & x > 0 \\ x - 1, & x < 0 \end{cases}$$

```
def eq1(x):  
    if x > 0:  
        return x + 1  
    elif x < 0:  
        return x - 1
```

```
y = eq1(0) # y= None
```

```
def eq1(x):  
    if x > 0:  
        output_value = x + 1  
    elif x < 0:  
        output_value = x - 1  
    return output_value
```

```
y = eq1(0) # Error
```

How to check None

x is None

x is not None

How to check None: Example



```
def purchasing Iphone(income=None):  
    if income is None:  
        return False  
  
    if income > 30000:  
        return True  
    else:  
        return False
```

Caution of Function

```
def func_1(x=None):  
    a = 0  
    b = 1
```

```
def func_2(x=None):  
    a = 1  
    b = 2
```

`a` in `func_1` is distinct
from `a` in `func_2`.

```
def func_1(x=None):  
    a = 0  
    b = 1
```

```
def func_2(x=None):  
    a = 1  
    b = 2
```

`func_2()` ✓

`func_1()` ✓

`func_2()` ✗

Hand-on 20 min without AI

```
def iphone_plan(  
    iphone_price: float,  
    monthly_income: float,  
    Installment_months: int | None = None):
```

```
    pass # do_something
```

```
    return n_month, monthly_payment, status
```

```
# status = "Affordable" or "Insufficient"
```

```
# months_needed : int, monthly_payment : float
```



Level : ★★☆☆☆